

Methods of Evaluating Estimators

Math 742

2026-02-17

Introduction

How do we decide if an estimator is good? There are 3 typical ways to evaluate estimators (Bias/Variance and MSE). To motivate this, I will use the analogy of shooting arrows at a target.

Bias: Have your friend shoot 10 arrows and for each arrow, measure how far it is from the bullseye. Now average all 10 of their shots. If their average distance from the bullseye is zero, the bias would be zero. If all the arrows are 6 inches to the right the bias would be high.

Variance: The variance, as you know, is a measure of spread. If your friend's arrows are in a tight clump the variance would be low, if they are all over the place the variance would be high.

Mean Squared Error (MSE): The MSE measures how far the arrows land from the bullseye overall. MSE combines both the bias and the variance. Your friend hopes that their shot pattern is near the bullseye (low bias) and tightly grouped (small variance). MSE combines these into one metric.

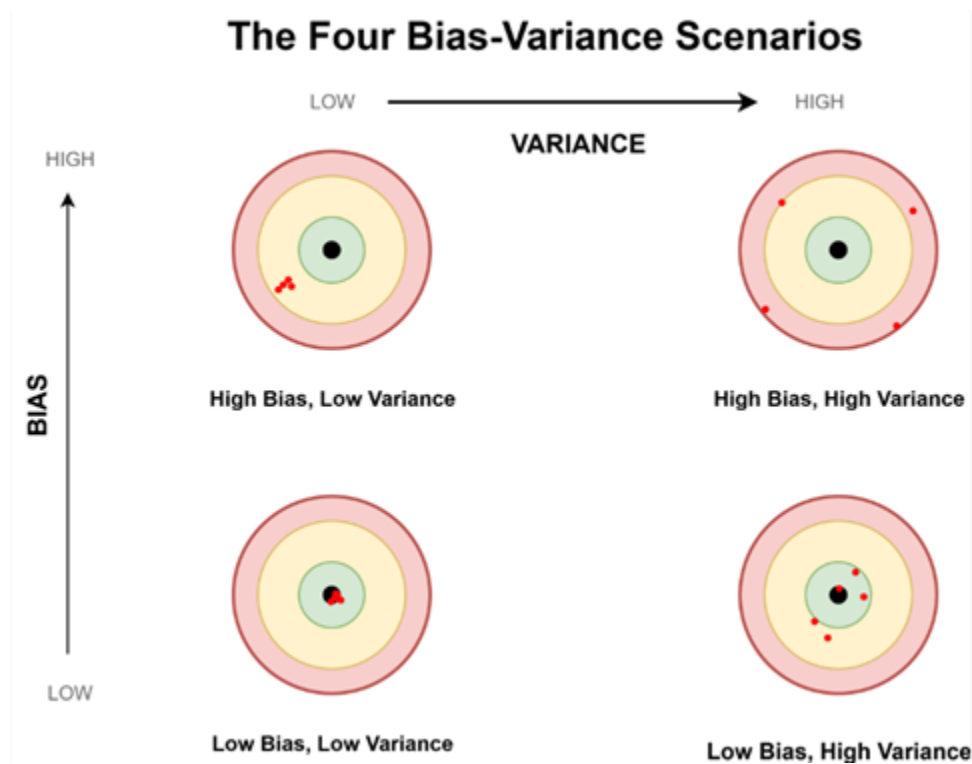


Figure 1: Bias/Variance

Mathematical Definitions:

Definition 1: Suppose we have a point estimator (i.e. single value) that is used to estimate a parameter θ . The **bias** is defined as:

$$\text{Bias}(\hat{\theta}) = \mathbb{E}(\hat{\theta}) - \theta$$

If $\mathbb{E}(\hat{\theta}) = \theta$, we say that $\hat{\theta}$ is an **unbiased** estimate of θ .

Definition 2: Suppose we have a point estimator ($\hat{\theta}$) that is used to estimate a parameter θ . The **Mean Squared Error (MSE)** is defined as:

$$\text{MSE}(\hat{\theta}) = \mathbb{E}(\hat{\theta} - \theta)^2$$

Notice that instead of squaring we could take the absolute value of the difference. One reason we don't do this is because the MSE has the property that it equals the sum of the variance and the squared bias.

For any estimator $\hat{\theta}$ of a parameter θ , the mean squared error satisfies

$$\begin{aligned}\text{MSE}(\hat{\theta}) &= \mathbb{E}_{\theta}[(\hat{\theta} - \theta)^2] = \mathbb{V}(\hat{\theta}) + (\mathbb{E}[\hat{\theta}] - \theta)^2. \\ \text{MSE} &= \text{Variance} + \text{Bias}^2\end{aligned}$$

Proof:

We begin by inserting and subtracting the expectation of $\hat{\theta}$:

$$\text{MSE}(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \theta)^2] = \mathbb{E}[(\hat{\theta} - \mathbb{E}[\hat{\theta}] + \mathbb{E}[\hat{\theta}] - \theta)^2].$$

Expanding the square yields

$$\mathbb{E}[(\hat{\theta} - \mathbb{E}[\hat{\theta}])^2 + 2(\hat{\theta} - \mathbb{E}[\hat{\theta}])(\mathbb{E}[\hat{\theta}] - \theta) + (\mathbb{E}[\hat{\theta}] - \theta)^2].$$

The middle term vanishes because

$$\mathbb{E}(\hat{\theta} - \mathbb{E}[\hat{\theta}]) = 0,$$

and $(\mathbb{E}[\hat{\theta}] - \theta)$ is constant with respect to the expectation.

Thus,

$$\text{MSE}(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \mathbb{E}[\hat{\theta}])^2] + (\mathbb{E}[\hat{\theta}] - \theta)^2.$$

Recognizing the terms as variance and squared bias gives

$$\text{MSE}(\hat{\theta}) = \mathbb{V}(\hat{\theta}) + (\text{Bias}(\hat{\theta}))^2$$

□

From a data science perspective, it's possible to find the “sweet” spot. The minimum MSE, but there is a tradeoff between bias and variance.

- **Model Flexibility:** A model's capacity to bend and adapt to the data's shape, capturing nonlinearities, interactions, and nuances. Think of a wiggly line vs. a straight line (less flexible).

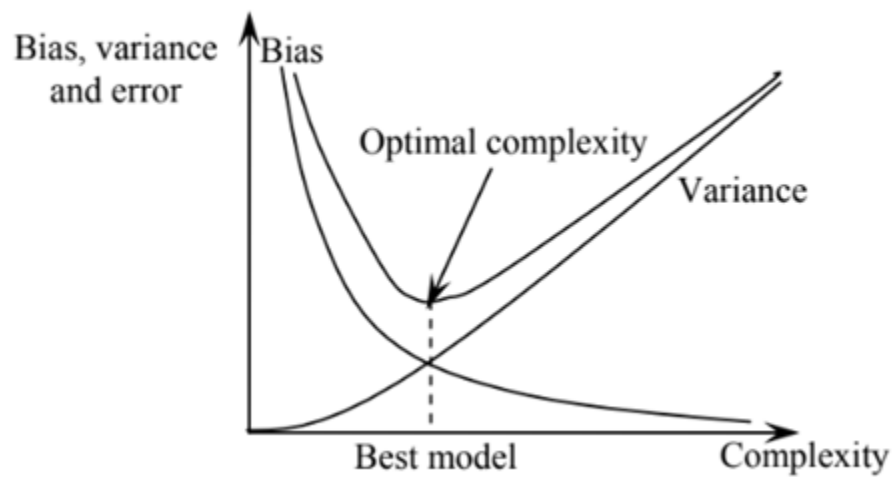


Figure 2: Bias Variance Tradeoff

- Model Complexity: The technical “size” or “degrees of freedom” of a model, often measured by the number of parameters or the richness of its functional form (e.g., high-degree polynomials, deep neural networks, many splines).

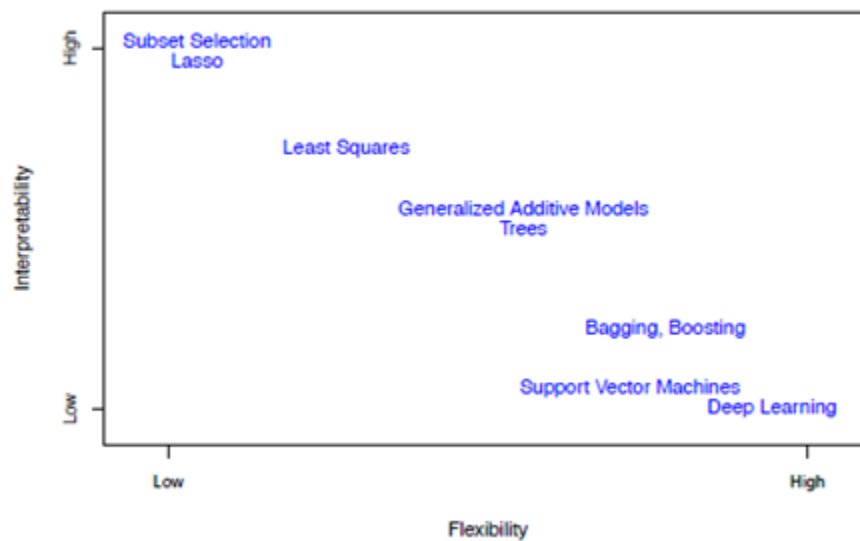


Figure 3: Interpretability vs. Flexibility

- Flexibility refers to how easily a model can adapt to different shapes in the data.

It answers: How wiggly can this model be?

Example: Let $X_1, X_2, \dots, X_n$ be an iid sample from $N(\mu, \sigma^2)$. The estimators $\bar{X}$ and S^2 are unbiased estimates of μ and σ^2 respectively.

Note: This proof is a repeat of the same proof that was done in the sampling distributions lecture FOR READING ONLY

Let $\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$ be the sample mean. By linearity of expectation and identical distribution,

$$\mathbb{E}[\bar{X}] = \frac{1}{n} \sum_{i=1}^n \mathbb{E}[X_i] = \frac{1}{n} \cdot n\mu = \mu,$$

so $\bar{X}$ is an unbiased estimator of μ .

Now consider the (uncentered) sum of squared deviations:

$$\sum_{i=1}^n (X_i - \bar{X})^2 = \sum_{i=1}^n [(X_i - \mu) - (\bar{X} - \mu)]^2.$$

Expanding and summing gives the identity

$$\sum_{i=1}^n (X_i - \bar{X})^2 = \sum_{i=1}^n (X_i - \mu)^2 - n(\bar{X} - \mu)^2.$$

Take expectations of both sides. Since the X_i are iid with variance σ^2 ,

$$\mathbb{E} \left[\sum_{i=1}^n (X_i - \mu)^2 \right] = \sum_{i=1}^n \mathbb{V}(X_i) = n\sigma^2,$$

and

$$\mathbb{E}[(\bar{X} - \mu)^2] = \mathbb{V}(\bar{X}) = \frac{\sigma^2}{n}.$$

Therefore,

$$\mathbb{E} \left[\sum_{i=1}^n (X_i - \bar{X})^2 \right] = n\sigma^2 - n \cdot \frac{\sigma^2}{n} = n\sigma^2 - \sigma^2 = (n-1)\sigma^2.$$

Finally, since $S^2 = (n-1)^{-1} \sum_{i=1}^n (X_i - \bar{X})^2$,

$$\mathbb{E}[S^2] = \frac{1}{n-1} \mathbb{E} \left[\sum_{i=1}^n (X_i - \bar{X})^2 \right] = \frac{(n-1)\sigma^2}{n-1} = \sigma^2.$$

Thus S^2 is an unbiased estimator of σ^2 .

Now, consider the MLE estimate of σ^2 . We've shown earlier that this is biased.

$$\hat{\sigma}^2 = \frac{1}{n} \sum_{i=1}^n (X_i - \bar{X})^2 = \frac{n-1}{n} S^2$$

$$\mathbb{E}[\hat{\sigma}^2] = \frac{n-1}{n}\sigma^2 \implies \text{Bias}(\hat{\sigma}^2) = -\frac{\sigma^2}{n}$$

We have shown the S^2 is unbiased, so $\mathbb{E}(S^2) = \sigma^2$

$$\mathbb{V}(\hat{\sigma}^2) = \mathbb{V}\left(\frac{n-1}{n}S^2\right) = \left(\frac{n-1}{n}\right)^2 \mathbb{V}(S^2) = \left(\frac{n-1}{n}\right)^2 \left(\frac{2\sigma^4}{n-1}\right) = \frac{2(n-1)\sigma^4}{n^2}$$

Note: The proof that $\mathbb{V}(S^2) = \left(\frac{2\sigma^4}{n-1}\right)$ is a little complicated. It relies on the fact that the sum of squared normal random variables follows a chi-squared distribution, and the variance of a chi-squared random variable is $2k$ where k is the degrees of freedom of the chi-squared distribution.

So, the MSE of $\hat{\sigma}^2$ is,

$$\begin{aligned} \mathbb{E}(\hat{\sigma}^2 - \sigma^2)^2 &= \mathbb{V}(\hat{\sigma}^2) + [\text{Bias}(\hat{\sigma}^2)]^2 \\ &= \frac{2(n-1)\sigma^4}{n^2} + \left(\frac{n-1}{n}\sigma^2 - \sigma^2\right)^2 = \left(\frac{2n-1}{n^2}\right)\sigma^4 \end{aligned}$$

This shows us that,

$$\mathbb{E}(\hat{\sigma}^2 - \sigma^2)^2 = \left(\frac{2n-1}{n^2}\right)\sigma^4 < \frac{2}{n-1}\sigma^4 = \mathbb{E}(S^2 - \sigma^2)^2$$

Which shows is that $\hat{\sigma}^2$ has smaller MSE than S^2 . So, by trading off variance for bias, the MSE improved.

Conclusion: Although S^2 is an unbiased estimator of σ^2 , the MLE estimator $\hat{\sigma}^2 = \frac{1}{n}(X_i - \bar{X})^2$ has smaller mean squared error. The reason is that $\hat{\sigma}^2$ introduces a small negative bias—shrinking the estimate slightly toward zero—but in doing so it reduces variance enough that the overall MSE improves. Since MSE reflects total estimation error (both variance and squared bias), an estimator with small bias and substantially lower variance can outperform an unbiased estimator. Thus, while S^2 is unbiased, the MLE $\hat{\sigma}^2$ is actually “better” under the MSE criterion. This illustrates an important general principle: **unbiasedness is not always the optimal goal, sometimes a little bias leads to much more accurate estimators overall.**

For normal models, both the MoM (and MLE) estimator of σ^2 is equal to $\frac{1}{n} \sum_{i=1}^n (X_i - \bar{X})^2$. As we’ve seen, this estimator is slightly biased. In contrast, the sample variance S^2 is unbiased, but it has higher variance. By using the MoM/MLE estimator, we trade a small amount of bias for reduced variance, resulting in a smaller MSE, which can be a better option overall.

Desirable Properties of Estimators

- **Unbiasedness:** Unbiasedness ensures that, on average, an estimator hits the true parameter, so repeated samples do not systematically over or under-estimate it.

$$\mathbb{E}(\hat{\theta}) = \theta$$

- **Consistency:** Consistency guarantees that as the sample size grows, the estimator converges to the true value, providing reliability in large samples.

$$\hat{\theta}_n \xrightarrow{p} \theta$$

- **Efficiency:** Efficiency means the estimator achieves (or asymptotically achieves) the Cramér–Rao lower bound *within a given model and under regularity conditions*, giving the most precise estimates.

$$\mathbb{V}(\hat{\theta}) = \frac{1}{I(\theta)}, \text{ Achieves Cramér–Rao lower bound}$$

- **Asymptotic normality:** Asymptotic normality allows inference using normal approximations even when exact distributions are complicated or unknown. An estimator is **asymptotically normal** if its scaled error converges in distribution to a normal distribution as $n \rightarrow \infty$

$$\sqrt{n}(\hat{\theta}_n - \theta) \xrightarrow{d} N(0, V)$$

→ where V is the asymptotic variance of the estimator, not the variance of the underlying data.

*Essentially, asymptotic normality is a statement about the distribution of the estimator, not about the data.&

- **Robustness:** Robustness means an estimator remains reliable even when assumptions are slightly wrong—such as mild deviations from the model, heavy tails, or a few outliers. A robust estimator does not change drastically when the data contain unexpected values or when the distribution is misspecified. This makes it valuable in real-world applications where ideal model conditions rarely hold perfectly.

Unbiasedness isn’t the “Be all end all”

Consider these examples.

Example: Let $X_1, X_2, \dots, X_n$ be an iid random sample from a $\text{Uniform}(0, \theta)$ distribution.

The MoM estimator is $2\bar{X}$ and the MLE estimator is $X_{(n)}$ the max order statistic. Compare the bias, variance and MSE of these estimators.

```
set.seed(9711)

# True parameter
theta <- 10

# Simulation settings
n <- 20          # sample size
B <- 10000       # number of Monte Carlo replicates

# Storage
mom <- numeric(B)
mle <- numeric(B)

for (b in 1:B) {
  x <- runif(n, 0, theta)

  # Method of Moments estimator
  mom[b] <- 2 * mean(x)

  # Maximum Likelihood estimator
  mle[b] <- max(x)
}

# Compute bias, variance, MSE
bias <- c(MoM = mean(mom) - theta,
```

```

MLE = mean(mle) - theta)

variance <- c(MoM = var(mom),
             MLE = var(mle))

mse <- c(MoM = mean((mom - theta)^2),
        MLE = mean((mle - theta)^2))

# Put results in a neat table
results <- rbind(Bias = bias,
                 Variance = variance,
                 MSE = mse)

round(results, 4)

```

```

##           MoM      MLE
## Bias      -0.0114 -0.4787
## Variance   1.6654  0.2139
## MSE        1.6653  0.4431

```

Example: Consider the linear model,

$$y = X\beta + \varepsilon, \text{ where } \varepsilon \sim N(0, \sigma^2 I)$$

The Gauss-Markov theorem tells us that the ordinary least squares (OLS) regression coefficients are unbiased.

$$\hat{\beta}_{OLS} = (X^T X)^{-1} X^T y$$

Later on we will discuss Ridge regression, the ridge regression coefficients are biased.

$$\hat{\beta}_{ridge} = (X^T X + \lambda I)^{-1} X^T y$$

The great thing about ridge regression is that it massively reduces the variance, especially when your predictor variables are correlated.

Consider the following R simulation:

```

set.seed(2025)

# simulation parameters
n <- 50
p <- 5
rho <- 0.95          # high correlation
Sigma <- matrix(rho, p, p); diag(Sigma) <- 1

library(MASS)

beta_true <- c(3, 0, 0, 0, 0)
B <- 5000

# store estimates
ols_est <- matrix(0, B, p)
ridge_est <- matrix(0, B, p)

```

```

lambda <- 5 # moderate ridge penalty

for (b in 1:B) {
  X <- mvrnorm(n, mu = rep(0, p), Sigma = Sigma)
  eps <- rnorm(n, 0, 1)
  y <- X %*% beta_true + eps

  # OLS
  ols_est[b, ] <- solve(t(X) %*% X) %*% t(X) %*% y

  # Ridge
  ridge_est[b, ] <- solve(t(X) %*% X + lambda * diag(p)) %*% t(X) %*% y
}

# Function to compute bias, variance, and MSE
compute_stats <- function(est, beta_true) {
  bias <- colMeans(est) - beta_true
  variance <- apply(est, 2, var)
  mse <- bias^2 + variance
  data.frame(Bias = bias, Variance = variance, MSE = mse)
}

# Compute statistics for OLS and Ridge
ols_stats <- compute_stats(ols_est, beta_true)
ols_stats <- round(ols_stats, 4)
ridge_stats <- compute_stats(ridge_est, beta_true)

# Combine results in one table
results <- rbind(OLS = ols_stats, Ridge = ridge_stats)
ridge_stats <- round(ridge_stats, 4)
results

```

```

##           Bias  Variance    MSE
## OLS.1   -0.0091000 0.36260000 0.3627000
## OLS.2   -0.0060000 0.36530000 0.3653000
## OLS.3    0.0083000 0.36400000 0.3641000
## OLS.4    0.0058000 0.36940000 0.3695000
## OLS.5    0.0016000 0.36510000 0.3652000
## Ridge.1 -1.6457863 0.04698329 2.7555960
## Ridge.2  0.3917944 0.04050059 0.1940035
## Ridge.3  0.3972141 0.04043815 0.1982172
## Ridge.4  0.3980269 0.04144330 0.1998687
## Ridge.5  0.3938628 0.04124211 0.1963700

```

The regression models have 5 coefficients (i.e. 5 predictor variables). What you are seeing in the table is the bias, variance and MSE for each coefficient.

- OLS is unbiased but has high variance under multi-collinearity.
- Ridge introduces bias through shrinkage but greatly stabilizes the estimator.
- The variance reduction outweighs the bias, so the ridge estimator almost always has lower MSE and better prediction accuracy.
- James–Stein (1961) proved that for $p \geq 3$, the MLE/OLS is inadmissible under MSE. In other words, there exists a biased estimator that dominates it everywhere.

Consistency

An estimator $\hat{\theta}_n$ is **consistent** for θ if

$$\hat{\theta}_n \xrightarrow{p} \theta \quad \text{as } n \rightarrow \infty.$$

Intuitively: with enough data, $\hat{\theta}_n$ gets arbitrarily close to the truth with probability $\rightarrow 1$.

Two Key Examples

1. Sample mean $\bar{X} \xrightarrow{p} \mu$

By the **Weak Law of Large Numbers** (WLLN) + Chebyshev:

$$\text{Var}(\bar{X}) = \frac{\sigma^2}{n} \Rightarrow \mathbb{P}(|\bar{X} - \mu| > \varepsilon) \leq \frac{\sigma^2}{n\varepsilon^2} \rightarrow 0.$$

2. Sample variance $S^2 \xrightarrow{p} \sigma^2$

Rewrite:

$$S^2 = \frac{n}{n-1} \left[\frac{1}{n} \sum (X_i - \mu)^2 - (\bar{X} - \mu)^2 \right]$$

Now apply convergence:

- $\frac{1}{n} \sum (X_i - \mu)^2 \xrightarrow{p} \sigma^2$ (WLLN on X_i^2 , assuming $\mathbb{E}[X^2] < \infty$)
- $\bar{X} \xrightarrow{p} \mu \Rightarrow (\bar{X} - \mu)^2 \xrightarrow{p} 0$
- $\frac{n}{n-1} \rightarrow 1$

Thus $S^2 \xrightarrow{p} \sigma^2$.

$$\boxed{\bar{X} \xrightarrow{p} \mu, \quad S^2 \xrightarrow{p} \sigma^2 \quad (\text{both are consistent})}$$

Takeaway: Consistency is the bare minimum we expect from any serious estimator. If it's not consistent, throw it away, no amount of bias-variance finesse will save it as $n \rightarrow \infty$.

Table of Common Estimators

Dist	θ	Est.	Unb?	MLE?	Notes
$N(\mu, \sigma^2)$	μ	$\bar{X}$	Yes	Yes	Minimum-variance unbiased estimator (MVUE)
$N(\mu, \sigma^2)$	σ^2	$S^2 = \frac{1}{n-1} \sum (X_i - \bar{X})^2$	Yes	No	Higher MSE than MLE
$N(\mu, \sigma^2)$	σ^2	$\hat{\sigma}^2 = \frac{1}{n} \sum (X_i - \bar{X})^2$	No	Yes	Biased but lower MSE; more efficient than S^2
Bernoulli(p)	p	$\hat{p} = \bar{X}$	Yes	Yes	MSE = $p(1-p)/n$
Exponential(λ)	λ	$\hat{\lambda} = 1/\bar{X}$	No	Yes	Asymptotically unbiased; consistent

Takeaway – Bias-Variance Tradeoff is Everything

In high dimensions or under model uncertainty, deliberately biased estimators almost always dominate unbiased ones under MSE or prediction error.

Examples: Ridge/Lasso/Elastic Net, James–Stein, Bayesian shrinkage, dropout, weight decay, early stopping...

All of modern statistical learning is built on controlled bias injection to slash variance.

Criteria for Comparing Estimators

In practice — especially in statistical learning and high-dimensional modeling, we rarely declare one estimator “best” using a single property (e.g., unbiasedness). Instead, we compare them under several complementary criteria:

- **Mean Squared Error (MSE):** The gold standard for point estimation: $\text{MSE}(\hat{\theta}) = \text{Var}(\hat{\theta}) + \text{Bias}^2(\hat{\theta})$. Directly measures total estimation error.
- **Asymptotic variance:** For large n , consistent estimators are compared by their limiting variance (or rate: $O(n^{-1})$, $O(n^{-1/2})$, etc.). Example: Ridge vs OLS when $p \approx n$.
- **Integrated risk / expected loss:** Decision-theoretic view: $R(\hat{\theta}, \theta) = \mathbb{E}_{\theta}[L(\hat{\theta}, \theta)]$. MSE is the risk under squared-error loss; log-loss, 0-1 loss, etc. are common alternatives.
- **Likelihood-based criteria (when applicable):** Among candidate estimators, the MLE maximizes the likelihood by construction. Profile likelihood, AIC, BIC, and focused information criteria extend this idea.
- **Prediction risk (out-of-sample error)** The ultimate goal in machine learning: how well $\hat{\theta}$ helps predict new data. Cross-validation estimates this directly.

Takeaway – There is no universally "best" estimator.

- Unbiasedness is rarely the goal in modern data science.
- We almost always trade a little bias for a lot less variance (ridge, lasso, dropout, early stopping, Bayes, etc.).
- The dominant criterion is **out-of-sample predictive risk** — everything else (MSE, asymptotic variance, likelihood) is a proxy for it.

Why This Matters for Data Science PhDs

- Hyperparameter tuning (λ in ridge/lasso) = choosing how much bias to inject
- Cross-validation = empirically estimating out-of-sample MSE
- The entire regularization revolution (1990s–today) is just “make it biased but lower variance”
- Test-set MSE, log-loss, AUC, etc. are all just generalizations of the MSE criterion

Python Code

```
import numpy as np

np.random.seed(9711)
```

```

# True parameter
theta = 10

# Simulation settings
n = 20          # sample size
B = 10000       # number of Monte Carlo replicates

# Storage
mom = np.zeros(B)
mle = np.zeros(B)

for b in range(B):
    # Generate n uniform random variables from U(0, theta)
    x = np.random.uniform(0, theta, n)

    # Method of Moments estimator: 2 * mean(x)
    mom[b] = 2 * np.mean(x)

    mle[b] = np.max(x)

# Compute bias, variance, MSE
bias_mom = np.mean(mom) - theta
bias_mle = np.mean(mle) - theta

var_mom = np.var(mom)
var_mle = np.var(mle)

mse_mom = np.mean((mom - theta)**2)
mse_mle = np.mean((mle - theta)**2)

# Create results table
results = np.array([
    [bias_mom, bias_mle],
    [var_mom, var_mle],
    [mse_mom, mse_mle]
])

# Round to 4 decimal places
results_rounded = np.round(results, 4)

# Display nicely
import pandas as pd
df = pd.DataFrame(results_rounded,
                  index=['Bias', 'Variance', 'MSE'],
                  columns=['MoM', 'MLE'])
print(df)

##           MoM      MLE
## Bias      0.0272 -0.4767
## Variance  1.6665  0.2004
## MSE       1.6673  0.4277

# OLS vs Ridge simulation - corrected version (works in R Markdown + reticulate)
import numpy as np

```

```

import pandas as pd

np.random.seed(2025)

# Simulation parameters
n = 50
p = 5
rho = 0.95
beta_true = np.array([3.0, 0.0, 0.0, 0.0, 0.0])

# Covariance matrix (exactly the same as R's matrix(rho,p,p); diag=1)
Sigma = np.full((p, p), rho)
np.fill_diagonal(Sigma, 1.0)

B = 5000
lambda_ridge = 5.0

# Storage
ols_est = np.zeros((B, p))      # <-- fixed name
ridge_est = np.zeros((B, p))    # <-- fixed name

for b in range(B):
    # Multivariate normal data
    X = np.random.multivariate_normal(np.zeros(p), Sigma, size=n)

    # Error and response
    eps = np.random.standard_normal(n)
    y = X @ beta_true + eps

    # OLS
    XtX = X.T @ X
    ols_est[b] = np.linalg.solve(XtX, X.T @ y)                # <-- ols_est

    # Ridge
    ridge_est[b] = np.linalg.solve(XtX + lambda_ridge * np.eye(p),
                                    X.T @ y)                  # <-- ridge_est

# Helper function
def compute_stats(estimates, true_beta):
    bias = estimates.mean(axis=0) - true_beta
    var = estimates.var(axis=0)
    mse = bias**2 + var
    return pd.DataFrame({
        'Bias': np.round(bias, 4),
        'Variance': np.round(var, 4),
        'MSE': np.round(mse, 4)
    })

# Results
ols_stats = compute_stats(ols_est, beta_true)
ridge_stats = compute_stats(ridge_est, beta_true)

# Combine with proper labeling

```

```

results = pd.concat({'OLS': ols_stats, 'Ridge': ridge_stats}, names=['Method'])

# Optional: nicer index names for the final table
results = results.reset_index(level=1, drop=True)
results.index = [f'Beta{i}' for i in range(p)] * 2
results = results.reset_index().rename(columns={'index': 'Coefficient'})
results = results[['Coefficient', 'Bias', 'Variance', 'MSE']]

results

```

##	Coefficient	Bias	Variance	MSE
## 0	Beta0	-0.0027	0.3472	0.3472
## 1	Beta1	0.0012	0.3603	0.3603
## 2	Beta2	0.0030	0.3535	0.3535
## 3	Beta3	-0.0034	0.3673	0.3673
## 4	Beta4	0.0034	0.3667	0.3667
## 5	Beta0	-1.6419	0.0450	2.7409
## 6	Beta1	0.3962	0.0409	0.1979
## 7	Beta2	0.3943	0.0397	0.1951
## 8	Beta3	0.3912	0.0424	0.1954
## 9	Beta4	0.3956	0.0411	0.1976